

전력사용량 예측 AI 경진대회

EDA STUDY

이름 : 배소연

데이터 탐색

1. train.csv

100개 건물들의 2022년 06월 01일부터 2022년 08월 24일까지의 데이터
일시별 기온, 강수량, 풍속, 습도, 일조, 일사 정보 포함
전력사용량(kWh) 포함

2. building_info.csv

100개 건물 정보
건물 번호, 건물 유형, 연면적, 냉방 면적, 태양광 용량, ESS 저장 용량, PCS 용량

3. test.csv

100개 건물들의 2022년 08월 25일부터 2022년 08월 31일까지의 데이터
일시별 기온, 강수량, 풍속, 습도의 예보 정보

4. sample_submission.csv

제출을 위한 양식

100개 건물들의 2022년 08월 25일부터 2022년 08월 31일까지의 전력사용량(kWh)을 예측
num_date_time은 건물번호와 시간으로 구성된 ID
해당 ID에 맞춰 전력사용량 예측값을 answer 컬럼에 기입해야 함

데이터 탐색 - building_info

컬럼명 영문 변경 필요

```
building_info.head(3)
```

✓ 0.0s Python

	건물번호	건물유형	연면적(m2)	냉방면적(m2)	태양광용량(kW)	ESS저장용량(kWh)	PCS용량(kW)
0	1	건물기타	110634.00	39570.0	-	-	-
1	2	건물기타	122233.47	99000.0	-	-	-
2	3	건물기타	171243.00	113950.0	40	-	-

-로 나타난 값들

데이터 탐색 - building_info

```
building_info = building_info.rename(columns={
    '건물번호': 'building_number',
    '건물유형': 'building_type',
    '연면적(m2)': 'total_area',
    '냉방면적(m2)': 'cooling_area',
    '태양광용량(kW)': 'solar_power_capacity',
    'ESS저장용량(kWh)': 'ess_capacity',
    'PCS용량(kW)': 'pcs_capacity'
})
```

✓ 0.0s

Python

```
building_info.isna().sum()
```

✓ 0.0s

Python

building_number	0
building_type	0
total_area	0
cooling_area	0
solar_power_capacity	0
ess_capacity	0
pcs_capacity	0
dtype: int64	

컬럼명 변경 완료, 결측치 따로 없으므로 -으로 표시된 값은 해당 사항 없음을 의미한다고 추론 가능

데이터 탐색 - building_info

빌딩 타입 값들 영문 변경

```
building_info['building_type'].unique()
```

✓ 0.0s

Python

```
array(['건물기타', '공공', '대학교', '데이터센터', '백화점및아울렛', '병원', '상용', '아파트', '연구소',  
      '지식산업센터', '할인마트', '호텔및리조트'], dtype=object)
```

```
translation_dict = {  
    '건물기타': 'Other Buildings',  
    '공공': 'Public',  
    '대학교': 'University',  
    '데이터센터': 'Data Center',  
    '백화점및아울렛': 'Department Store and Outlet',  
    '병원': 'Hospital',  
    '상용': 'Commercial',  
    '아파트': 'Apartment',  
    '연구소': 'Research Institute',  
    '지식산업센터': 'Knowledge Industry Center',  
    '할인마트': 'Discount Mart',  
    '호텔및리조트': 'Hotel and Resort'  
}
```

```
building_info['building_type'] = building_info['building_type'].replace(translation_dict)
```

✓ 0.0s

Python

데이터 탐색 - train_df

num_date_time = 건물번호 + 일시

```
train_df.head(3)
```

✓ 0.0s Python

	num_date_time	건물번호	일시	기온 (C)	강수량 (mm)	풍속 (m/s)	습도 (%)	일조 (hr)	일사 (MJ/m2)	전력소비량 (kWh)
0	1_20220601 00	1	20220601 00	18.6	NaN	0.9	42.0	NaN	NaN	1085.28
1	1_20220601 01	1	20220601 01	18.0	NaN	1.1	45.0	NaN	NaN	1047.36
2	1_20220601 02	1	20220601 02	17.7	NaN	1.5	45.0	NaN	NaN	974.88

train_df가 갖는 독특한 변수 일조/일사/전력소비량

데이터 탐색 - train_df

컬럼명 영문 변경

```
train_df = train_df.rename(columns={
    '건물번호': 'building_number',
    '일시': 'date_time',
    '기온(C)': 'temperature',
    '강수량(mm)': 'rainfall',
    '풍속(m/s)': 'windspeed',
    '습도(%)': 'humidity',
    '일조(hr)': 'sunshine',
    '일사(MJ/m2)': 'solar_radiation',
    '전력소비량(kWh)': 'power_consumption'
})

train_df.drop('num_date_time', axis = 1, inplace=True)
```

✓ 0.0s

Python

num_date_time 컬럼 드랍

데이터 탐색 - train_df

```
train_df.isna().sum()
```

✓ 0.0s

Python

building_number	0
date_time	0
temperature	0
rainfall	160069
windspeed	19
humidity	9
sunshine	75182
solar_radiation	87913
power_consumption	0
dtype: int64	

결측값 확인

- rainfall/sunshine/solar_radiation 에 결측치 매우 많음
- windspeed, humidity 결측치 소수 존재

데이터 탐색 - test_df

num_date_time = 건물번호 + 일시

```
test_df.head(3)
```

✓ 0.0s

Python

	num_date_time	건물번호	일시	기온(C)	강수량(mm)	풍속(m/s)	습도(%)
0	1_20220825 00	1	20220825 00	23.5	0.0	2.2	72
1	1_20220825 01	1	20220825 01	23.0	0.0	0.9	72
2	1_20220825 02	1	20220825 02	22.7	0.0	1.5	75

데이터 탐색 - test_df

컬럼명 영문 변경

```
test_df = test_df.rename(columns={
    '건물번호': 'building_number',
    '일시': 'date_time',
    '기온(C)': 'temperature',
    '강수량(mm)': 'rainfall',
    '풍속(m/s)': 'windspeed',
    '습도(%)': 'humidity',
    '일조(hr)': 'sunshine',
    '일사(MJ/m2)': 'solar_radiation',
    '전력소비량(kWh)': 'power_consumption'
})

test_df.drop('num_date_time', axis = 1, inplace=True)
```

✓ 0.0s

Python

num_date_time 컬럼 드랍

데이터 탐색 - test_df

결측치 없음

```
test_df.isna().sum()
```

✓ 0.0s

Python

```
building_number    0  
date_time          0  
temperature        0  
rainfall           0  
windspeed          0  
humidity           0  
dtype: int64
```

생각해보아야 할 점

1. building_info에서 '-' 으로 표시된 값은 측정치가 없는 값
(null 이 아니라 해당 수치가 존재하지 않는 값)
2. train_df 의 결측치 처리 방법

전처리 - building_info

```
building_info['solar_power_device'] = building_info['solar_power_capacity'] != '-'  
building_info['ess_power_device'] = building_info['ess_capacity'] != '-'  
building_info['pcs_power_device'] = building_info['pcs_capacity'] != '-'
```

✓ 0.0s

Python

각각 capacity가 - 이면 장치가 설치되지 않아 측정값이 없음으로 보고,
device 설치 유무에 따른 컬럼 생성하여
장치가 존재하면 True, 존재하지 않으면 False로 처리

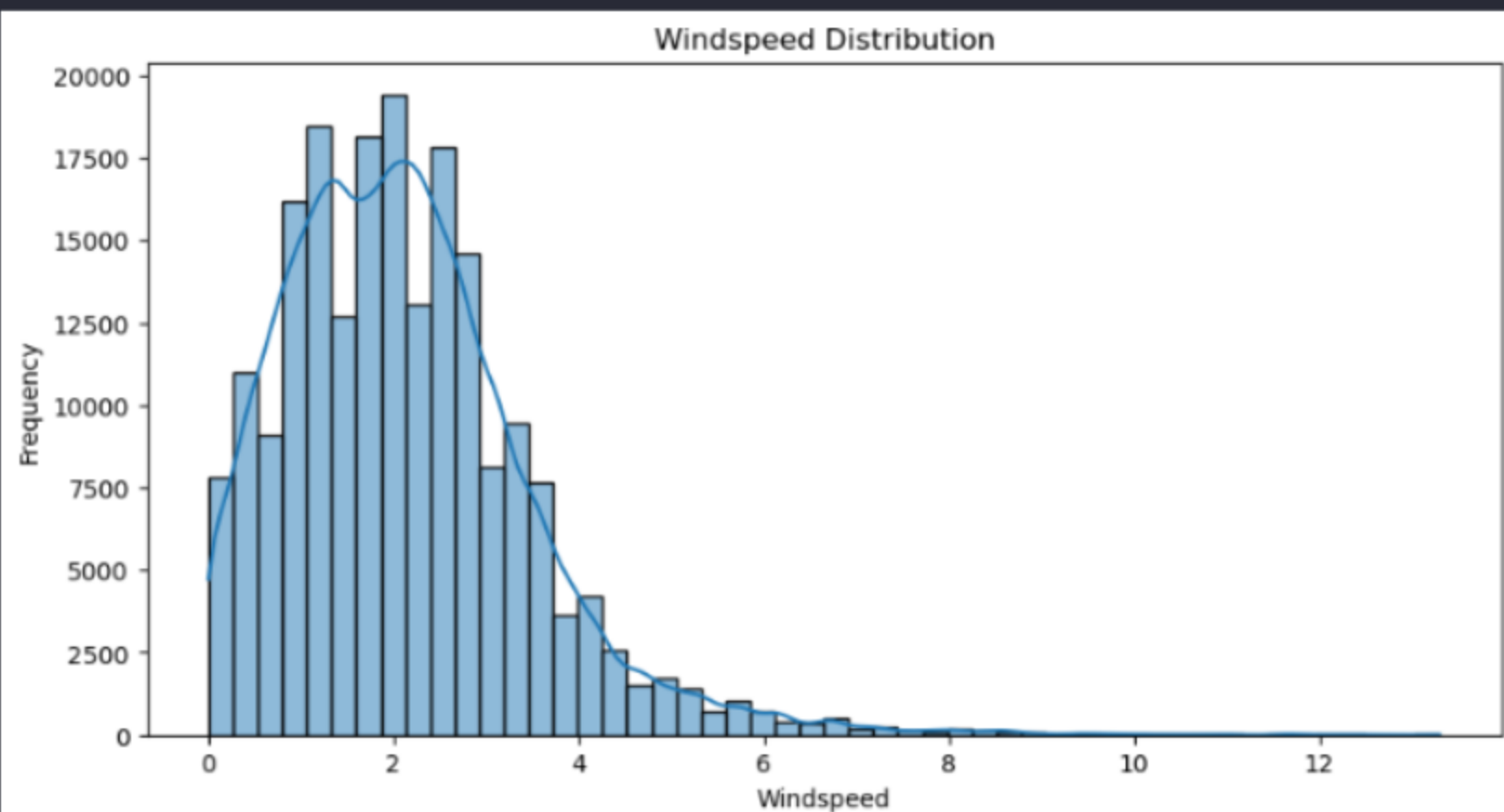
02

전처리 - train_df

```
plt.figure(figsize=(10, 5))
sns.histplot(train_df['windspeed'], bins=50, kde=True)
plt.title('Windspeed Distribution')
plt.xlabel('Windspeed')
plt.ylabel('Frequency')
plt.show()
```

✓ 0.8s

Python

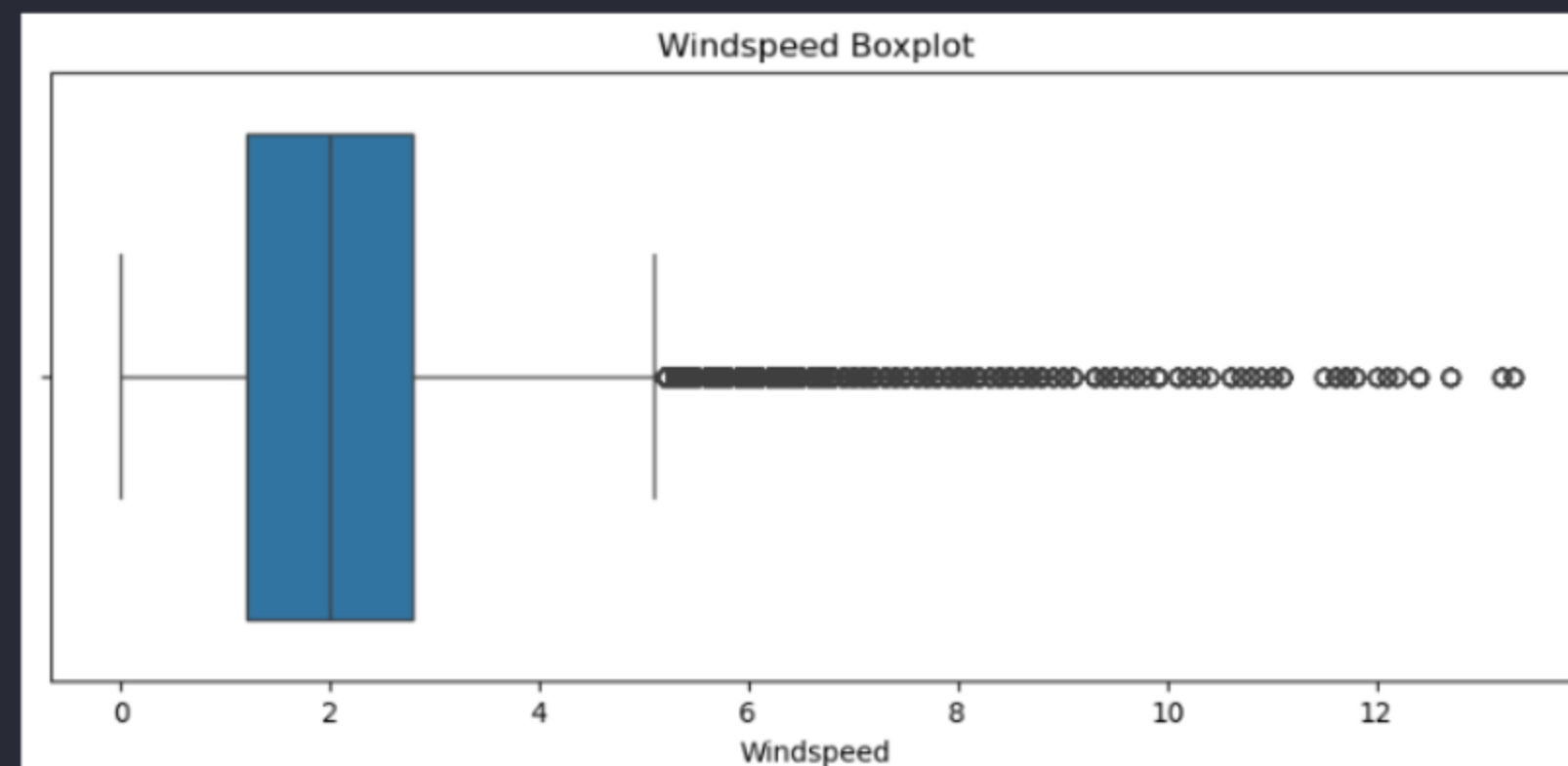


```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 4))
sns.boxplot(x=train_df['windspeed'])
plt.title('Windspeed Boxplot')
plt.xlabel('Windspeed')
plt.show()
```

✓ 0.2s

Python



전처리 - train_df

```
mean_windspeed = train_df['windspeed'].mean()  
train_df['windspeed'] = train_df['windspeed'].fillna(mean_windspeed)
```

✓ 0.0s

Python

```
train_df['windspeed'].describe()
```

✓ 0.0s

Python

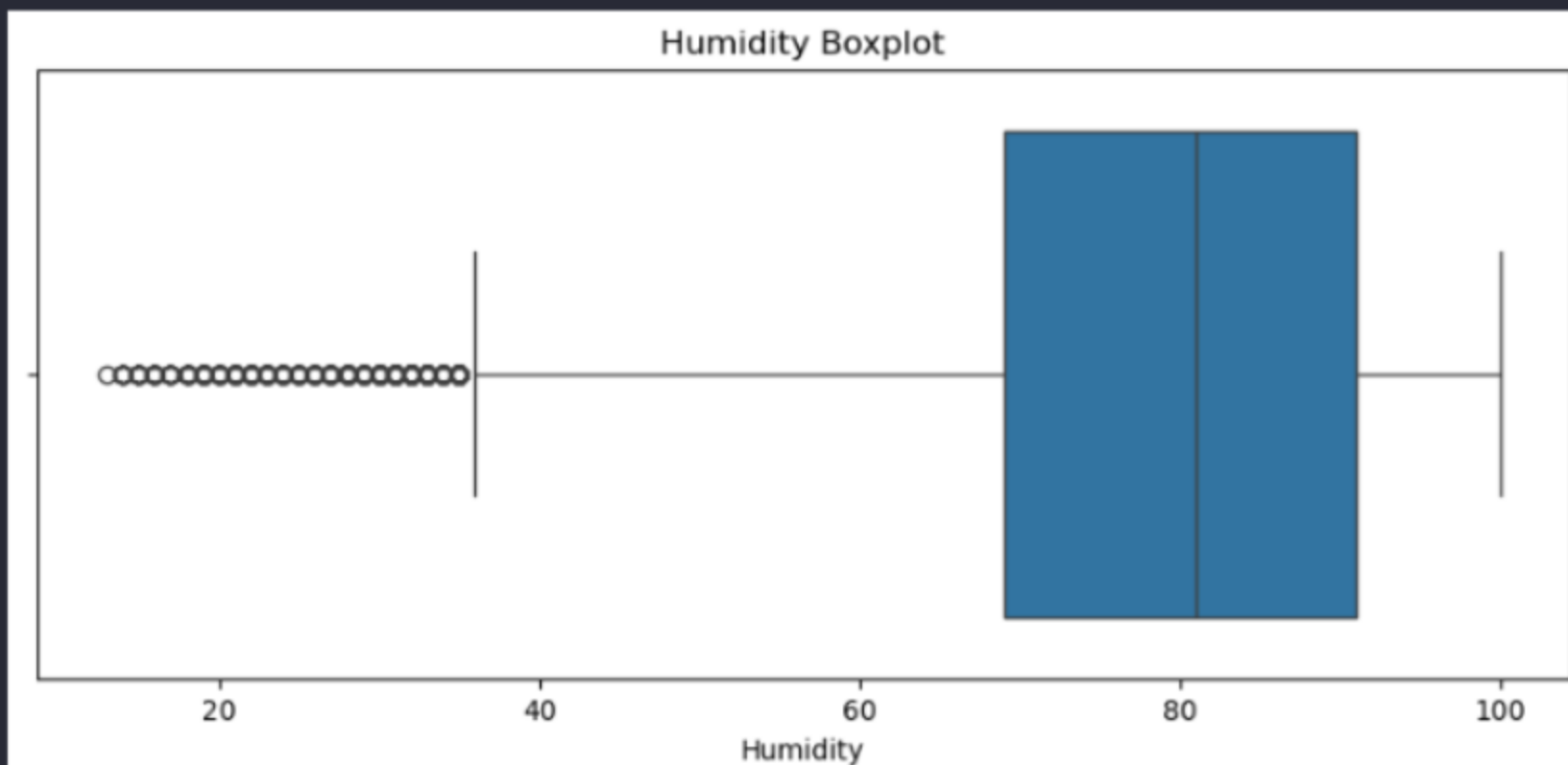
```
count    204000.000000  
mean         2.134031  
std         1.358045  
min         0.000000  
25%         1.200000  
50%         2.000000  
75%         2.800000  
max        13.300000  
Name: windspeed, dtype: float64
```


전처리 - train_df

```
plt.figure(figsize=(10, 4))
sns.boxplot(x=train_df['humidity'])
plt.title('Humidity Boxplot')
plt.xlabel('Humidity')
plt.show()
```

✓ 0.2s

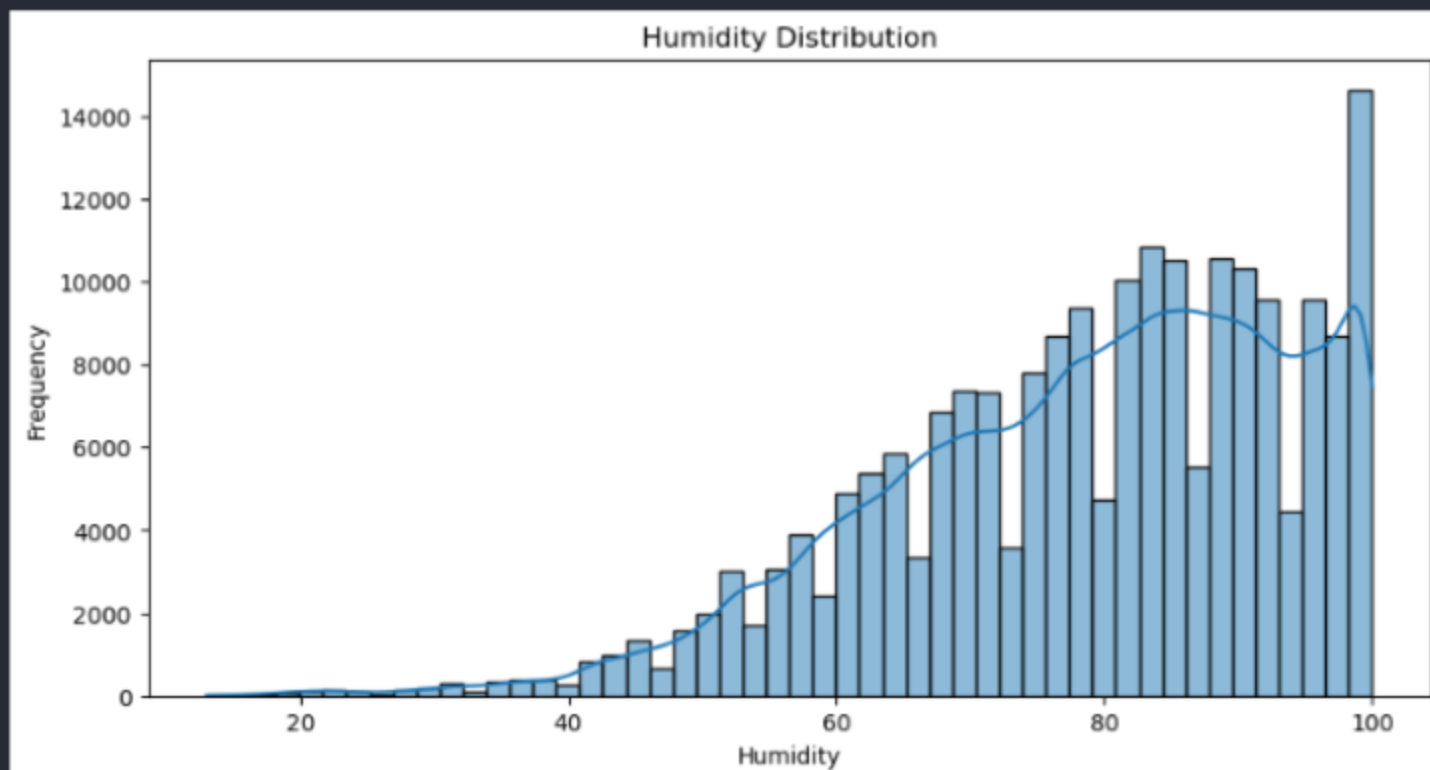
Python



```
plt.figure(figsize=(10, 5))
sns.histplot(train_df['humidity'], bins=50, kde=True)
plt.title('Humidity Distribution')
plt.xlabel('Humidity')
plt.ylabel('Frequency')
plt.show()
```

✓ 0.6s

Python



전처리 - train_df

```
mean_humidity = train_df['humidity'].mean()  
train_df['humidity'] = train_df['humidity'].fillna(mean_humidity)
```

✓ 0.0s

Python

```
train_df['humidity'].describe()
```

✓ 0.0s

Python

```
count    204000.000000  
mean       78.716295  
std       15.079775  
min       13.000000  
25%       69.000000  
50%       81.000000  
75%       91.000000  
max       100.000000  
Name: humidity, dtype: float64
```

전처리 - train_df

```
# 시간 기준 정렬
train_df = train_df.sort_values(by='date_time')

# 선형 보간 적용 (결측치 많은 변수)
cols_to_interpolate = ['rainfall', 'sunshine', 'solar_radiation']

for col in cols_to_interpolate:
    train_df[col] = train_df[col].interpolate(method='linear', limit_dir
```

✓ 0.0s

Python

결측치가 너무 많은 변수는 평균으로 적용하기 어려움

→ 시계열 데이터이니 선형 보간 적용

전처리 - train_df

```
# 1. date_time을 datetime 형식으로 변환
train_df['date_time'] = pd.to_datetime(train_df['date_time'], format='%Y%m%d %H')

# 2. 연도, 월, 일, 요일, 시간 컬럼 분리
train_df['year'] = train_df['date_time'].dt.year
train_df['month'] = train_df['date_time'].dt.month
train_df['day'] = train_df['date_time'].dt.day
train_df['weekday'] = train_df['date_time'].dt.weekday
train_df['hour'] = train_df['date_time'].dt.hour
```

✓ 0.0s

Python

date_time이 year, month, day, hour 합쳐져 있음

→ year, month, day, hour 각각으로 분류

→ 추가로 요일도 datetime 객체를 이용해 파생 변수 생성

전처리 - train_df

```
holiday_list = [  
    '2022-01-01', '2022-01-31', '2022-02-01', '2022-02-02', '2022-03-01', '2022-03-09',  
    '2022-05-05', '2022-05-08', '2022-06-01', '2022-06-06', '2022-08-15', '2022-09-09',  
    '2022-09-10', '2022-09-11', '2022-09-12', '2022-10-03', '2022-10-09', '2022-10-10',  
    '2022-12-25'  
]  
  
holiday_dates = pd.to_datetime(holiday_list)  
✓ 0.0s Python
```

```
# 주말이면 holiday 컬럼에 True, 아니면 False로 설정  
train_df['holiday'] = (train_df['date_time'].dt.weekday >= 5) | (train_df['date_time'].isin(holiday_dates))
```

휴일을 의미하는 holiday 변수 → 주말, 공휴일 True, 평일 False

전처리 - train_df

```
def add_weather_indicators(train_df):
    # 불쾌지수 (Temperature-Humidity Index)
    train_df['THI'] = (
        (9 / 5) * train_df['temperature']
        - 0.55 * (1 - train_df['humidity'] / 100) * ((9 / 5) * train_df['temperature'] - 26)
        + 32
    )

    # 체감온도 (Wind Chill)
    train_df['WC'] = (
        13.12 + 0.6215 * train_df['temperature']
        - 13.947 * (train_df['windspeed']**0.16)
        + 0.486 * train_df['temperature'] * (train_df['windspeed']**0.16)
    )

    # CDH (Cumulative Degree Hours)
    def calculate_cdh(temps, window=12, threshold=26):
        # 초과 온도 계산 (음수는 0으로 처리)
        excess = np.maximum(temps - threshold, 0)
        # 누적합 슬라이딩 윈도우 방식
        cumsum = np.cumsum(np.insert(excess, 0, 0))
        return cumsum[window:] - cumsum[:-window]

    # 건물별로 CDH 계산
    cdhs = []
    for bnum in train_df['building_number'].unique():
        temp_data = train_df[train_df['building_number'] == bnum].copy()
        temps = temp_data['temperature'].values
        # 패딩 처리해서 원래 길이 유지
        cdh = calculate_cdh(temps)
        cdh_padded = np.concatenate([np.zeros(11), cdh]) # window-1 만큼 0 패딩
        cdhs.extend(cdh_padded)

    train_df['CDH'] = cdhs

    return train_df
```

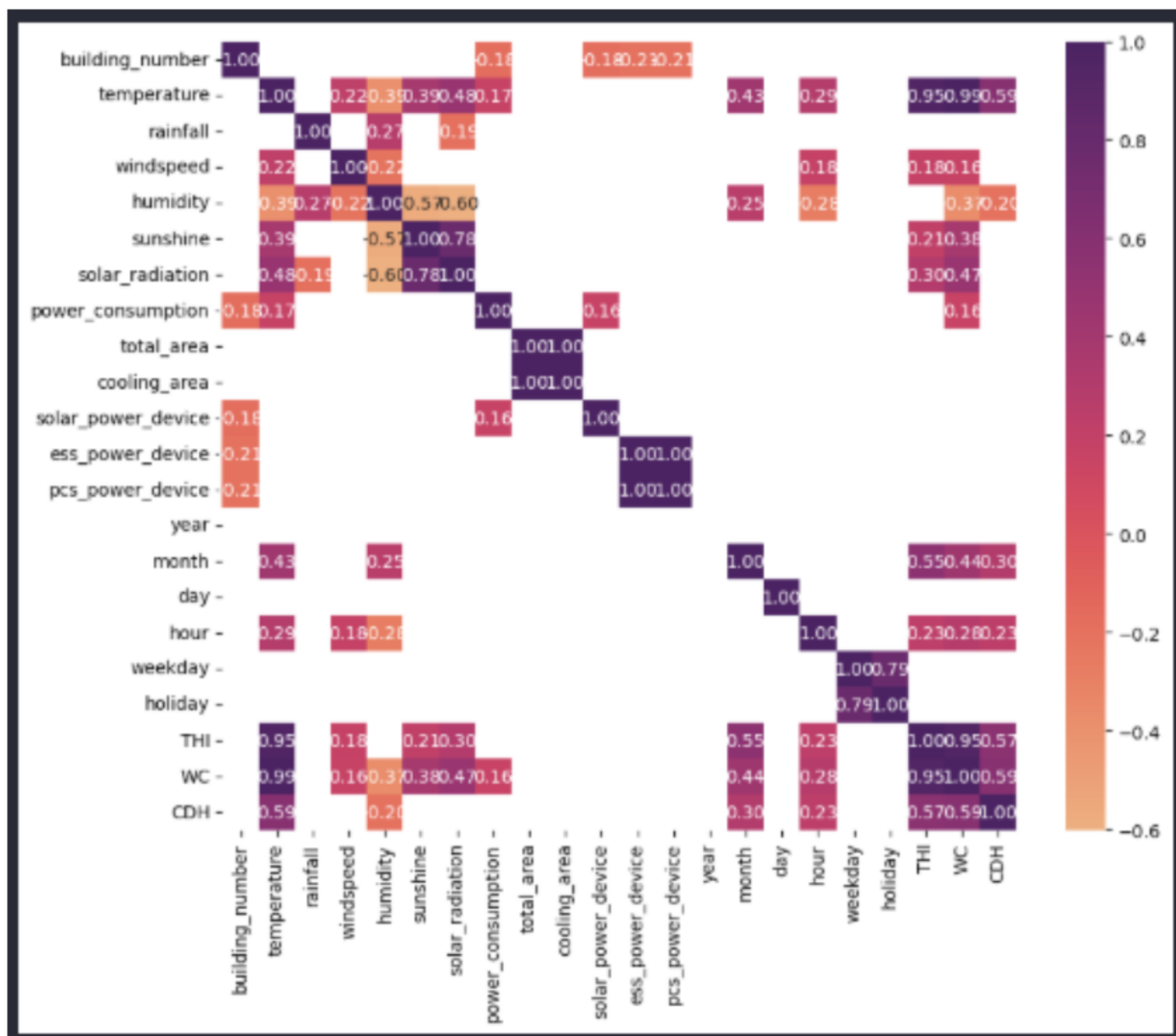
불쾌지수 (THI)

체감온도 (Wind Chill, WC)

누적 냉방도 (CDH)

→ 건물별 12시간 누적 온도 계산 (기준 26도 이상)

상관계수 분석



전력소비와 관련있는 변수

- 건물 타입, 온도, 태양열 장치 여부, 불쾌지수

느낀 점

1. 결측치 보간을 어떻게 할 것인가?

→ 일단 평균값으로 채우기는 했지만, 날씨의 다른 요소를 고려하여 채우는 방법도 있었을 것이고, 날짜를 범위로 분류해서 좀 더 평균값을 세심하게 구할 수도 있었을 것이다.
어떤 방법이 좀 더 정확도를 높일 수 있을지 더 세심한 방법을 적용해야 한다

2. 파생 변수

→ 지금은 불쾌지수, 체감온도, 누적 냉방도, 휴일 등의 파생변수를 만들었다
그러나 휴일의 경우 건물 타입에 따라 달라질 수 있고, 어떤 휴일은 좀 더 특별한 휴일 (장기, 명절 등) 일 수도 있는데 이런 경우를 가중치를 준다면 할 수 있을 듯
또한 누적 냉방도 이런 것도 면적이 넓으면 냉방을 더 오래해야 좁은 곳과 비슷하게 내려갈 수 있는데 이런 경우들을 고려하지 않았다.